EuroHPC Development Access Proposal

PyHPC-Fin: Massively Parallel Option Pricing with Python on EuroHPC

Group 8

1. Abstract & Objectives

Abstract:

Financial risk management requires real-time pricing of complex derivatives, a task often demanding thousands of compute hours. While specialized C++/FPGA solutions exist, they lack the flexibility needed for rapid quantitative research. This project, PyHPC-Fin, aims to bridge the gap between high-productivity languages (Python) and high-performance hardware (EuroHPC Exascale systems). We propose to evolve an existing MPI-based CPU Monte Carlo engine into a GPU-accelerated, massively parallel framework capable of scaling to thousands of GPUs on LUMI-G or Leonardo.

Objectives:

1. **GPU Porting**: Transition the core Monte Carlo kernel from CPU/NumPy to GPU-accelerated Python (using CuPy/Numba/HIP) to leverage the massive throughput of EuroHPC accelerators.
2. **Exascale Scaling**: Demonstrate "weak scaling" efficiency >80% on up to 1,024 GPUs, enabling pricing of massive portfolios in seconds.
3. **Energy Efficiency**: Quantify the energy-per-option-priced advantage of GPU vs. CPU architectures, contributing to "Green Finance" initiatives.

2. State of the Art (SoTA)

Current Landscape: High-frequency trading firms utilize FPGAs and optimized C++ for nanosecond latency. However, risk management (calculating Greeks, VaR) requires throughput, not just latency. Most banks run large on-premise CPU grids (Grid computing). There is a lack of open-source, scalable Python frameworks that natively leverage modern GPU supercomputers without requiring Quants to write C++/CUDA.

Project Innovation: PyHPC-Fin provides a "pure Python" interface while handling MPI communication and GPU kernels under the hood. It democratizes access to Exascale power for financial researchers using standard tools (pip install, numpy syntax).

3. Current Code & TRL

- **Code Status**: The current codebase is a fully functional MPI + NumPy implementation
- **Architecture**: SPMD (Single Program Multiple Data) using mpi4py.
- **Algorithm**: Geometric Brownian Motion (Black-Scholes) for European Options.
- **Optimization**: Vectorized operations and Antithetic Variates for variance reduction.
- **Validation**: Rigorously tested against analytical Black-Scholes solutions; convergence of *O(N^{-0.5})* verified.
- **Performance**: Demonstrated ~85% parallel efficiency on 6-node CPU clusters.
- **Technology Readiness Level (TRL)**: Current TRL: 4 (Technology validated in a laboratory environment). The code runs correctly and scales on small clusters. The goal of this access is to reach TRL 6/7 (System prototype demonstration in an operational environment) by deploying on EuroHPC pre-exascale systems.

4. Target EuroHPC Machine & Stack

We request access to LUMI (Finland), specifically the LUMI-G (GPU) partition.

Justification: LUMI-G is based on AMD MI250X GPUs. This architecture is crucial for our goal of proving that Python-based HPC is vendor-agnostic (supporting both CUDA and ROCm ecosystems).

Target Stack:

- Languages: Python 3.10+
- Communication: cray-mpich (GPU-aware MPI)
- Computation: cupy (for Nvidia/AMD agnostic arrays) or numba (generating ROCm/CUDA kernels).
- Containerization: Singularity/Apptainer for environment reproducibility.

5. Work Plan

Duration: 12 Months

Work Package 1: GPU Kernel Porting (Months 1-3)

- Task 1.1: Replace numpy array operations with cupy/jax/numba to offload computation to GPUs.

- Task 1.2: Implement memory management strategies to handle large batches that exceed GPU VRAM (streaming).
- Milestone: Single-GPU speedup of >50x vs Single-CPU core.

Work Package 2: Multi-GPU Comm. Optimization (Months 4-6)

- Task 2.1: Integrate GPU-aware MPI (Direct GPU-to-GPU memory transfer, bypassing CPU RAM).
- Task 2.2: Optimize reduction operations (MPI_Reduce) for minimal latency.
- Milestone: Multi-GPU scaling efficiency >70% on 4 nodes (16 GPUs).

Work Package 3: Large-Scale Benchmarking (Months 7-12)

- Task 3.1: Weak scaling runs on 100, 500, and 1000+ GPUs.
- Task 3.2: Scientific validation of random number generation quality at scale (ensuring no period exhaustion).
- Milestone: Final report and open-source release.

6. Resource Justification

Request: 50,000 GPU Node Hours.

Calculation:

- **Development & Debugging**: 200 hours (interactive functionality checks).
- **Scaling Tests (WP2)**: 4 nodes * 24h * 10 runs = ~1,000 hours.
- **Large Scale Runs (WP3)**:

  100 nodes * 1 hour * 10 runs = 1,000 hours.

  500 nodes * 0.5 hour * 5 runs = 1,250 hours.

- **Contingency**: ~20% for failed jobs/queue times.

This allocation is necessary because standard university clusters lack the specific MI250X architecture and the high-speed interconnect (Slingshot 11) required to test the interconnect bottlenecks.

7. Data Management (FAIR) & Ethics

Data Policy: The project uses synthetic data (randomly generated Monte Carlo paths). No personal or proprietary data is processed.

- **Findable/Accessible**: All code and benchmark logs will be hosted on GitHub.
- **Interoperable**: Output formats are standard CSV/HDF5.
- **Reusable**: MIT License allows free academic and commercial reuse.

Ethics: This research involves no human subjects, dual-use technology, or sensitive data. It contributes to financial stability research but has no direct military application.

8. Expected Impact

1. **Scientific**: Demonstration of Python's viability for Exascale computing, reducing the barrier to entry for domain scientists.
2. **Industrial**: Provides FinTech startups a blueprint to utilize cloud-HPC resources for accurate risk pricing without massive R&D teams.
3. **Societal**: More efficient risk calculation leads to more stable financial markets and reduced energy consumption for banking datacenters.